

Lab Assignment

C17 — STL Algorithms — Core Operations

sort • find • find_if • count • count_if • copy • copy_if • transform • for_each • accumulate • min_element
• max_element

1. Objective

This lab introduces the most important algorithms in the C++ Standard Template Library. STL algorithms are **generic, reusable functions** that operate on any container through iterators. Rather than writing loops from scratch for common tasks like sorting, searching, filtering, and transforming data, you call a single well-tested algorithm — one line replaces ten.

All experiments use the Springfield Power Plant employees as data. Each algorithm is demonstrated on a `vector<Employee>` or a `vector<int>`.

By the end of this lab you will be able to:

- ▶ Sort a vector with `sort()` using default and custom comparators (lambdas).
- ▶ Search for elements with `find()` and `find_if()`.
- ▶ Count matching elements with `count()` and `count_if()`.
- ▶ Copy elements into a new container with `copy()` and filter during copy with `copy_if()`.
- ▶ Apply a function to every element with `for_each()`.
- ▶ Transform elements into a new container with `transform()`.
- ▶ Compute aggregates with `accumulate()`, `min_element()`, and `max_element()`.

2. Background

2.1 The Iterator-Based Design

Every STL algorithm takes a pair of iterators — `begin` and `end` — that define the range to operate on. This design means the same algorithm works on any container: vector, list, set, or even a raw array.

```
// All three calls use the same sort() algorithm:
sort(v.begin(), v.end());           // sort a vector
sort(arr, arr+n);                   // sort a raw array
sort(list.begin(), list.end());     // sort a list
```

2.2 Lambdas as Comparators and Predicates

Many algorithms accept a **comparator** (for sorting/searching) or a **predicate** (a function returning bool). The cleanest way to supply these is with a **lambda expression**:

```
// Lambda comparator -- sort by salary descending
sort(v.begin(), v.end(),
    [](const Employee& a, const Employee& b) {
        return a.getSalary() > b.getSalary();
    });

// Lambda predicate -- find first employee over 50
auto it = find_if(v.begin(), v.end(),
    [](const Employee& e) { return e.getAge() > 50; });
```

3. Algorithm - Quick Reference

All algorithms require `#include <algorithm>`. `accumulate` requires `#include <numeric>`.

Algorithm	Description
<code>sort(b, e)</code>	Sorts elements in [b,e) in ascending order.
<code>sort(b, e, cmp)</code>	Sorts using custom comparator cmp.
<code>find(b, e, val)</code>	Returns iterator to first element == val, or e if not found.
<code>find_if(b, e, pred)</code>	Returns iterator to first element where pred(e) is true, or e.
<code>count(b, e, val)</code>	Returns number of elements == val.
<code>count_if(b, e, pred)</code>	Returns number of elements where pred(e) is true.
<code>copy(b, e, out)</code>	Copies all elements from [b,e) to the output iterator out.
<code>copy_if(b, e, out, pred)</code>	Copies only elements where pred(e) is true.
<code>for_each(b, e, fn)</code>	Calls fn(element) for every element in [b,e).
<code>transform(b, e, out, fn)</code>	Applies fn to each element, writes result to out.
<code>accumulate(b, e, init)</code>	Sums all elements starting from init. Returns total.
<code>min_element(b, e)</code>	Returns iterator to the smallest element in [b,e).
<code>max_element(b, e)</code>	Returns iterator to the largest element in [b,e).
<code>any_of(b, e, pred)</code>	Returns true if pred is true for AT LEAST ONE element.
<code>all_of(b, e, pred)</code>	Returns true if pred is true for ALL elements.
<code>none_of(b, e, pred)</code>	Returns true if pred is true for NO elements.

4. Required Experiments

All experiments share the same employee dataset. Define it once at the top of your file:

```
// Shared dataset -- add this struct at file scope
struct Employee {
    string name;
    int    age;
    double salary;
    string dept;
};

vector<Employee> makeDB() {
    return {
        { "Homer Simpson",      45, 52000, "Sector 7-G" },
        { "Lenny Leonard",     46, 54000, "Sector 7-G" },
        { "Carl Carlson",      47, 68000, "Sector 7-G" },
        { "Frank Grimes",      38, 58000, "Engineering" },
        { "Professor Frink",    58, 88000, "Research" },
        { "Montgomery Burns", 104, 1000000, "Executive" },
        { "Waylon Smithers",    52, 95000, "Executive" },
        { "Hans Moleman",       77, 22000, "Maintenance" },
        { "Ned Flanders",      60, 58000, "Safety" },
        { "Groundskeeper Willie", 50, 34000, "Maintenance" },
    };
}
```

Experiment 01 — sort()

`sort()` rearranges elements in place. Without a comparator it uses `operator<`. With a lambda comparator you can sort by any field in any direction.

Tasks:

- ▶ Sort a `vector<int>` of ages in ascending order using default `sort()`.
- ▶ Sort employees by **salary descending** using a lambda comparator.
- ▶ Sort employees by **age ascending** using a lambda comparator.
- ▶ Sort employees by **name alphabetically** using a lambda comparator.
- ▶ Sort employees by **department, then name** within department.

```
void experiment01(vector<Employee> db) {
    cout << "=== Experiment 01: sort() ===\n";

    // Sort a plain vector<int>
    vector<int> ages = { 45, 46, 47, 38, 58, 104, 52, 77, 60, 50 };
    sort(ages.begin(), ages.end());
    cout << "Ages ascending: ";
    for (int a : ages) cout << a << " "; cout << "\n";

    // Sort employees by salary descending
    sort(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            return a.salary > b.salary;
        });
    cout << "\nBy salary (highest first):\n";
    for (const auto& e : db)
        cout << "  $" << e.salary << "  " << e.name << "\n";

    // Sort by age ascending
    sort(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            return a.age < b.age;
        });
    cout << "\nBy age (youngest first):\n";
    for (const auto& e : db)
        cout << "  age=" << e.age << "  " << e.name << "\n";

    // Sort by department then name within department
    sort(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            if (a.dept != b.dept) return a.dept < b.dept;
            return a.name < b.name;
        });
    cout << "\nBy department then name:\n";
    for (const auto& e : db)
        cout << "  [" << e.dept << "]  " << e.name << "\n";
}
```

Experiment 02 — find() and find_if()

`find()` searches for an exact value. `find_if()` searches for the first element satisfying a predicate — far more flexible.

Tasks:

- ▶ Use `find()` on a `vector<int>` to locate the value 58.
- ▶ Use `find_if()` to find the first employee older than 50.
- ▶ Use `find_if()` to find the first employee in the Safety department.
- ▶ Use `find_if()` to find the first employee earning more than \$80,000.
- ▶ Handle the case where the search finds nothing — check against `end()`.

```
void experiment02(const vector<Employee>& db) {
    cout << "=== Experiment 02: find() and find_if() ===\n";

    // find() on a plain vector<int>
    vector<int> ages = { 45, 46, 47, 38, 58, 104, 52, 77, 60, 50 };
    auto it = find(ages.begin(), ages.end(), 58);
    if (it != ages.end())
        cout << "find(58): found at index " << (it - ages.begin()) << "\n";
    else
        cout << "find(58): not found\n";

    // find_if() -- first employee older than 50
    auto it2 = find_if(db.begin(), db.end(),
        [](const Employee& e) { return e.age > 50; });
    if (it2 != db.end())
        cout << "First employee age > 50: " << it2->name
            << " (age " << it2->age << ")\n";

    // find_if() -- first employee in Safety
    auto it3 = find_if(db.begin(), db.end(),
        [](const Employee& e) { return e.dept == "Safety"; });
    if (it3 != db.end())
        cout << "First Safety employee: " << it3->name << "\n";
    else
        cout << "No Safety employees found.\n";

    // find_if() -- first employee earning > $80,000
    auto it4 = find_if(db.begin(), db.end(),
        [](const Employee& e) { return e.salary > 80000; });
    if (it4 != db.end())
        cout << "First earning > $80K: " << it4->name
            << " $" << it4->salary << "\n";
}
```

Experiment 03 — count() and count_if()

`count()` counts exact matches. `count_if()` counts elements satisfying a predicate. Both return an integer — no iterator handling needed.

Tasks:

- ▶ Count how many employees are in the Maintenance department.
- ▶ Count how many employees are older than 50.
- ▶ Count how many employees earn between \$50,000 and \$70,000.
- ▶ Use `any_of()`, `all_of()`, and `none_of()` for three quick membership checks.

```
void experiment03(const vector<Employee>& db) {
    cout << "=== Experiment 03: count() and count_if() ===\n";

    // count_if -- employees in Maintenance
    int maintenance = count_if(db.begin(), db.end(),
        [](const Employee& e) { return e.dept == "Maintenance"; });
    cout << "Maintenance employees: " << maintenance << "\n";

    // count_if -- employees older than 50
    int seniors = count_if(db.begin(), db.end(),
        [](const Employee& e) { return e.age > 50; });
    cout << "Employees age > 50:      " << seniors << "\n";

    // count_if -- salary in [50000, 70000]
    int midRange = count_if(db.begin(), db.end(),
        [](const Employee& e) {
            return e.salary >= 50000 && e.salary <= 70000;
        });
    cout << "Salary $50K-$70K:      " << midRange << "\n";

    // any_of / all_of / none_of
    bool anyOver100 = any_of(db.begin(), db.end(),
        [](const Employee& e) { return e.age > 100; });
    cout << "Any employee over 100: " << (anyOver100 ? "YES" : "NO") << "\n";

    bool allAdults = all_of(db.begin(), db.end(),
        [](const Employee& e) { return e.age >= 18; });
    cout << "All employees >= 18:    " << (allAdults ? "YES" : "NO") << "\n";

    bool noneUnpaid = none_of(db.begin(), db.end(),
        [](const Employee& e) { return e.salary == 0; });
    cout << "No unpaid employees:    " << (noneUnpaid ? "YES" : "NO") << "\n";
}
```

Experiment 04 — copy() and copy_if()

`copy()` copies all elements from one range to another. `copy_if()` copies only elements matching a predicate — this is the clean way to filter a container into a new one. Both write to an output iterator; use `back_inserter()` to append to a vector.

Tasks:

- ▶ Use `copy()` to duplicate the full employee list into a second vector.
- ▶ Use `copy_if()` to extract only Sector 7-G employees into a new vector.
- ▶ Use `copy_if()` to extract employees earning more than \$60,000.
- ▶ Use `copy()` with `ostream_iterator` to print a vector directly to `cout`.

```
#include <iterator>    // back_inserter, ostream_iterator

void experiment04(const vector<Employee>& db) {
    cout << "=== Experiment 04: copy() and copy_if() ===\n";

    // copy() -- full duplicate
    vector<Employee> backup;
    copy(db.begin(), db.end(), back_inserter(backup));
    cout << "backup size: " << backup.size() << "  (== db size)\n";

    // copy_if() -- Sector 7-G employees only
    vector<Employee> sector7g;
    copy_if(db.begin(), db.end(), back_inserter(sector7g),
        [](const Employee& e) { return e.dept == "Sector 7-G"; });
    cout << "\nSector 7-G employees:\n";
    for (const auto& e : sector7g)
        cout << "  " << e.name << "\n";

    // copy_if() -- high earners
    vector<Employee> highEarnings;
    copy_if(db.begin(), db.end(), back_inserter(highEarnings),
        [](const Employee& e) { return e.salary > 60000; });
    cout << "\nEmployees earning > $60,000:\n";
    for (const auto& e : highEarnings)
        cout << "  " << e.name << "  $" << e.salary << "\n";

    // copy() with ostream_iterator -- print ages directly
    vector<int> ages = { 45, 38, 58, 104, 52, 77 };
    cout << "\nAges via ostream_iterator: ";
    copy(ages.begin(), ages.end(),
        ostream_iterator<int>(cout, " "));
    cout << "\n";
}
```

Experiment 05 — for_each() and transform()

`for_each()` visits every element and calls a function on it — useful for printing, accumulating, or side effects. `transform()` applies a function to every element and writes the result to a new container, leaving the original unchanged.

Tasks:

- ▶ Use `for_each()` to print every employee's name and salary on one line each.
- ▶ Use `for_each()` with a stateful lambda to count Maintenance workers during traversal.
- ▶ Use `transform()` to extract all salaries into a `vector<double>`.
- ▶ Use `transform()` to build a `vector<string>` of formatted name + age strings.
- ▶ Apply a 10% raise to all salaries in place using `transform()` writing back to the same vector.

```
void experiment05(vector<Employee> db) {
    cout << "=== Experiment 05: for_each() and transform() ===\n";

    // for_each() -- print each employee
    cout << "Employee roster:\n";
    for_each(db.begin(), db.end(),
        [](const Employee& e) {
            cout << "    " << e.name
                << "    $" << e.salary << "\n";
        });

    // for_each() -- stateful lambda counts Maintenance workers
    int maintCount = 0;
    for_each(db.begin(), db.end(),
        [&maintCount](const Employee& e) {
            if (e.dept == "Maintenance") maintCount++;
        });
    cout << "\nMaintenance workers counted by for_each: " << maintCount << "\n";

    // transform() -- extract salaries into vector<double>
    vector<double> salaries;
    transform(db.begin(), db.end(), back_inserter(salaries),
        [](const Employee& e) { return e.salary; });
    cout << "\nSalaries extracted: ";
    for (double s : salaries) cout << s << " ";
    cout << "\n";

    // transform() -- build formatted strings
    vector<string> labels;
    transform(db.begin(), db.end(), back_inserter(labels),
        [](const Employee& e) {
            return e.name + " (age " + to_string(e.age) + ")";
        });
    cout << "\nFormatted labels:\n";
    for (const auto& lbl : labels) cout << "    " << lbl << "\n";

    // transform() -- 10% raise, written back in place
    transform(db.begin(), db.end(), db.begin(),
        [](Employee e) { e.salary *= 1.10; return e; });
    cout << "\nAfter 10% raise:\n";
    for (const auto& e : db)
        cout << "    $" << e.salary << "    " << e.name << "\n";
}
```

Experiment 06 — accumulate(), min_element(), max_element()

`accumulate()` computes a running total. `min_element()` and `max_element()` return iterators to the smallest and largest elements. All three accept custom comparators or binary operations.

Tasks:

- ▶ Use `accumulate()` to compute the total payroll (sum of all salaries).
- ▶ Use `accumulate()` with a custom operation to compute the total payroll excluding Burns.
- ▶ Use `min_element()` and `max_element()` to find the youngest and oldest employees.
- ▶ Use `min_element()` and `max_element()` with salary comparators to find lowest and highest paid.
- ▶ Compute the average salary from the total and count.

```
#include <numeric>      // accumulate

void experiment06(const vector<Employee>& db) {
    cout << "=== Experiment 06: accumulate, min/max_element ===\n";

    // accumulate() -- total payroll
    double totalPayroll = accumulate(db.begin(), db.end(), 0.0,
        [](double sum, const Employee& e) {
            return sum + e.salary;
        });
    cout << "Total payroll: $" << totalPayroll << "\n";

    // accumulate() -- total payroll excluding Burns
    double payrollNoBurns = accumulate(db.begin(), db.end(), 0.0,
        [](double sum, const Employee& e) {
            return (e.name != "Montgomery Burns") ? sum + e.salary : sum;
        });
    cout << "Payroll (ex Burns): $" << payrollNoBurns << "\n";

    // Average salary
    double avg = totalPayroll / db.size();
    cout << "Average salary:    $" << avg << "\n";

    // min/max age
    auto youngest = min_element(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            return a.age < b.age;
        });
    auto oldest = max_element(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            return a.age < b.age;
        });
    cout << "\nYoungest: " << youngest->name << " (age " << youngest->age <<
    ") \n";
    cout << "Oldest:    " << oldest->name << " (age " << oldest->age << ") \n";

    // min/max salary
    auto lowestPaid = min_element(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            return a.salary < b.salary;
        });
    auto highestPaid = max_element(db.begin(), db.end(),
        [](const Employee& a, const Employee& b) {
            return a.salary < b.salary;
        });
    cout << "Lowest paid: " << lowestPaid->name << " ($" << lowestPaid->salary << ") \n";
    cout << "Highest paid: " << highestPaid->name << " ($" << highestPaid->salary << ") \n";
}
```

```

    auto highestPaid = max_element(db.begin(), db.end(),
    [](const Employee& a, const Employee& b) {
        return a.salary < b.salary;
    });
    cout << "Lowest paid: " << lowestPaid->name
    << " $" << lowestPaid->salary << "\n";
    cout << "Highest paid: " << highestPaid->name
    << " $" << highestPaid->salary << "\n";
}

```

main() — Call All Experiments

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <numeric>
#include <iterator>
using namespace std;

int main() {
    vector<Employee> db = makeDB();

    experiment01(db);    // sort -- pass by value (sorts a copy)
    experiment02(db);    // find, find_if
    experiment03(db);    // count, count_if, any_of, all_of
    experiment04(db);    // copy, copy_if
    experiment05(db);    // for_each, transform -- pass by value
    experiment06(db);    // accumulate, min/max_element

    cout << "\nAll done!\n";
    return 0;
}

```

5. Expected Output (partial)

```
=== Experiment 01: sort() ===
Ages ascending: 38 45 46 47 50 52 58 60 77 104

By salary (highest first):
$1000000 Montgomery Burns
$95000 Waylon Smithers
$88000 Professor Frink ...

By department then name:
[Engineering] Frank Grimes
[Executive] Montgomery Burns
[Executive] Waylon Smithers ...

=== Experiment 02: find() and find_if() ===
find(58): found at index 4
First employee age > 50: Lenny Leonard (age 46) -- order depends on DB
First Safety employee: Ned Flanders
First earning > $80K: Professor Frink $88000

=== Experiment 03: count() and count_if() ===
Maintenance employees: 2
Employees age > 50: 5
Salary $50K-$70K: 4
Any employee over 100: YES
All employees >= 18: YES
No unpaid employees: YES

=== Experiment 04: copy() and copy_if() ===
backup size: 10 (== db size)
Sector 7-G employees:
Homer Simpson Lenny Leonard Carl Carlson
Employees earning > $60,000:
Carl Carlson $68000
Professor Frink $88000 ...

=== Experiment 05: for_each() and transform() ===
Maintenance workers counted by for_each: 2
Salaries extracted: 52000 54000 68000 58000 88000 ...

=== Experiment 06: accumulate, min/max_element ===
Total payroll: $1531000
Payroll (ex Burns): $531000
Average salary: $153100
Youngest: Frank Grimes (age 38)
Oldest: Montgomery Burns (age 104)
Lowest paid: Hans Moleman $22000
Highest paid: Montgomery Burns $1000000
```

Common Mistakes

Pitfalls to avoid

- ▶ **Not checking `find()` / `find_if()` result against `end()`** — if the element is not found, dereferencing the returned iterator is undefined behavior.
- ▶ **Forgetting `back_inserter()` with `copy_if()` and `transform()`** — the output vector must either be pre-sized or use `back_inserter()` to grow dynamically. Writing to an empty vector without it is undefined behavior.
- ▶ **Capturing variables by value in lambdas that modify state** — use `[&count]` (capture by reference) when the lambda needs to update a variable in the outer scope. `[count]` captures a copy and modifications are lost.
- ▶ **Sorting a container and then using stale iterators** — `sort()` invalidates all iterators and pointers into the vector. Re-obtain any iterators you need after a sort.
- ▶ **Passing `accumulate()` the wrong initial value type** — `accumulate(v.begin(), v.end(), 0)` uses integer arithmetic even for a `vector<double>`. Start with `0.0` to force floating-point accumulation.

7. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. What is the advantage of using `find_if()` over writing your own search loop? Give a concrete example.
- ▶ 2. Explain the difference between `for_each()` and `transform()`. When would you choose one over the other?
- ▶ 3. Why must you use `back_inserter()` with `copy_if()` when the output vector is empty?
- ▶ 4. Why does the lambda for `accumulate()` take two parameters while the lambda for `find_if()` takes only one?
- ▶ 5. You need to give every employee a 5% raise. Would you use `for_each()` or `transform()`? Explain the trade-offs of each approach.

8. Self-Assessment: Quality Control

Before *submitting* your work review each item below. A check on every line means your lab is complete.

☐	Item
☐	Experiment 01 — four sort orders (int, salary, age, dept+name) all produce correct output.
☐	Experiment 02 — <code>find()</code> and <code>find_if()</code> results always checked against <code>end()</code> before use.
☐	Experiment 03 — <code>count_if()</code> , <code>any_of()</code> , <code>all_of()</code> , <code>none_of()</code> all demonstrated.
☐	Experiment 04 — <code>copy()</code> and <code>copy_if()</code> use <code>back_inserter()</code> ; <code>ostream_iterator</code> demonstrated.
☐	Experiment 05 — <code>for_each()</code> with stateful lambda; <code>transform()</code> extracts, builds, and modifies in place.
☐	Experiment 06 — <code>accumulate()</code> uses 0.0 as initial value; <code>min/max_element</code> use comparator lambdas.
☐	Code compiles with no errors and no warnings.
☐	Lambdas capture by reference (<code>[&]</code>) only when they need to modify outer scope variables.
☐	No raw loops used where an STL algorithm would be cleaner.
☐	Reflection questions answered in 2–3 sentences each.

Key insight

Every raw loop you write to search, filter, sort, or aggregate data has a well-named, well-tested STL algorithm that does the same job. Learning to recognize "this loop is a `find_if`" or "this loop is a `transform`" is the transition from writing C++ to writing idiomatic C++. The algorithm name documents the intent; the lambda supplies the specifics.